SIMATS ENGINEERING
CO3-ASSESSMENT TOOL1-CASE STUDY

COURSE NAME: Artificial Intelligence **COURSE CODE:CSA1709**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS: 4 **Total Marks:40**

Code of Conduct:

I S DHANUSH (192524317) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S DHANUSH
192524317

Annexure A

CASE STUDY

Case Study 1 — Smart Medical Diagnosis System

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains:

I. If a patient has Fever AND Cough → Possible Flu

Ii. If a patient has Fever AND Rash → Possible Measles

Iii. If a patient has Cough AND Breathlessness → Possible Pneumonia

Iv. Facts about Patient A: Fever = True, Cough = True, Breathlessness = True

(a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.

(b) Apply Forward Chaining on the knowledge base to derive all possible diagnoses for Patient A. Show each inference step with the rule fired and fact derived.

(c) The goal is to confirm 'Patient A has Pneumonia.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

(a) Propositional Logic Representation

Let the propositions for **Patient A** be:

- F: Patient A has **Fever**
- C: Patient A has **Cough**
- R: Patient A has **Rash**
- B: Patient A has **Breathlessness**
- FL: Patient A has **Possible Flu**
- M: Patient A has **Possible Measles**
- P: Patient A has **Possible Pneumonia**

Rules

1. **Fever AND Cough → Possible Flu**

$(F \land C) \rightarrow FL$

2. **Fever AND Rash → Possible Measles**

$(F \land R) \rightarrow M$

3. **Cough AND Breathlessness → Possible Pneumonia**

$(C \land B) \rightarrow P$

Facts

The knowledge base contains:

F C B

Thus:

$KB = \{F, C, B, (F \land C) \rightarrow FL, (F \land R) \rightarrow M, (C \land B) \rightarrow P\}$

There is **no fact stating R** (Rash).

(b) Forward Chaining

Forward chaining starts with the known facts and repeatedly applies rules whose premises are satisfied.

Initial facts

F, C, B

That is:

- Fever = True
- Cough = True
- Breathlessness = True

Step 1: Apply Rule I

Rule I:

$(F \land C) \rightarrow FL$

Both F and C are known to be true.

Therefore:

FL

Derived: Patient A has Possible Flu.

Step 2: Apply Rule II

Rule II:

$(F \wedge R) \rightarrow M$

We know F, but **R is not known**.

Therefore, Rule II **cannot fire**.

No Measles diagnosis derived

Step 3: Apply Rule III

Rule III:

$(C \wedge B) \rightarrow P$

Both C and B are known to be true.

Therefore:

P

Derived: Patient A has Possible Pneumonia.

Forward-Chaining Summary

Step	Rule	Premises satisfied	New fact
Initial	—	F,C,B	Fever, Cough, Breathlessness
1	Rule I	F,C	FL — Possible Flu
2	Rule II	F, but no R	Nothing
3	Rule III	C,B	P — Possible Pneumonia

Therefore, all diagnoses that can be derived are:

Possible Flu

and

Possible Pneumonia

Measles cannot be derived because Rash is not a known fact.

(c) Backward Chaining for Pneumonia

We want to prove:

P

where P means **Patient A has Possible Pneumonia**.

Backward chaining starts with the goal and searches for a rule that can produce it.

Step 1: Find a rule whose conclusion is P

Rule III is:

$(C \land B) \rightarrow P$

Therefore, to prove P, we reduce the goal to:

$C \land B$

So the goal becomes two subgoals:

1. C: Patient A has Cough
2. B: Patient A has Breathlessness

Step 2: Verify C

C is already present as a fact:

$C = \text{True}$

Therefore, the first subgoal succeeds.

Step 3: Verify B

B is also already present as a fact:

$B = \text{True}$

Therefore, the second subgoal succeeds.

Goal Reduction Tree

```
                    Goal: P
          Patient A has Pneumonia
                       |
                Rule III
          (C ∧ B) → P
                       |
            ┌──────────┴──────────┐
            ↓                     ↓
      Goal: C               Goal: B
  Patient has Cough   Patient has Breathlessness
            |                     |
      Fact: C=True          Fact: B=True
            ✓                     ✓
            └──────────┬──────────┘
                       ↓
               P = True
           Pneumonia confirmed
```

Conclusion

Since both required subgoals C and B are facts in the knowledge base,

$C \wedge B = \text{True}$

and hence, by Rule III,

$P = \text{True}$

Therefore, **Backward Chaining successfully verifies that Patient A has Possible Pneumonia.**

Case Study 2 — Autonomous Traffic Management Agent

A smart city traffic controller uses a FOL-based knowledge base to manage emergency vehicle routing:

I. ∀x: Vehicle(x) ∧ EmergencyType(x) → ClearPath(x) [Rule 1]

II. ∀x: ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x) [Rule 2]

III. ∀x ∀y: Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x) [Rule 3]

IV. Facts: Vehicle(Ambulance), EmergencyType(Ambulance), Vehicle(CarA), Behind(CarA, Ambulance)

(a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set θ (MGU) at each step.

(b) Using the Resolution method, prove that 'Proceed (CarA)' is true. Convert relevant clauses to CNF and show each resolution step.

(c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency vehicles. Identify what additional rules or facts are needed and justify logically.

(a) Unification of Rule 1 with the Given Facts

Rule 1

$\forall x[\text{Vehicle}(x) \land \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)]$

For inference, we can write the rule as:

$\text{Vehicle}(x) \land \text{EmergencyType}(x) \rightarrow \text{ClearPath}(x)$

The relevant facts are:

$\text{Vehicle}(\text{Ambulance})\ \text{EmergencyType}(\text{Ambulance})$

Step 1: Unify Vehicle(x) with Vehicle(Ambulance)

We compare:

$\text{Vehicle}(x)$ with

$\text{Vehicle}(\text{Ambulance})$

The predicate and structure match, so:

$\theta 1=\{x/Ambulance\}$

After applying θ1, the first premise becomes:

Vehicle(Ambulance)

which is a known fact.

Step 2: Unify EmergencyType(x) with EmergencyType(Ambulance)

Using the same substitution:

$EmergencyType(x)\theta 1=EmergencyType(Ambulance)$

This matches the given fact exactly.

Thus:

$\theta 2=\{x/Ambulance\}$

Complete MGU

The most general unifier is therefore:

$\theta=\{x/Ambulance\}$

Applying this substitution to Rule 1:

$Vehicle(Ambulance) \wedge EmergencyType(Ambulance) \rightarrow ClearPath(Ambulance)$

Since both premises are true, we derive:

ClearPath(Ambulance)

Summary

Unification	MGU
Vehicle(x) with Vehicle(Ambulance)	{x/Ambulance}
EmergencyType(x) with EmergencyType(Ambulance)	{x/Ambulance}
Complete MGU	{x/Ambulance}

(b) Resolution Proof of Proceed(CarA)

We want to prove:

$\mathrm{Proceed}(\mathrm{CarA})$

Using **proof by contradiction**, add the negation of the goal:

$\mathrm{Proceed}(\mathrm{CarA})$

If this produces a contradiction, the goal is proven.

Step 1: Convert the relevant rules to CNF

Rule 1

Original:

$\mathrm{Vehicle}(x) \land \mathrm{EmergencyType}(x) \rightarrow \mathrm{ClearPath}(x)$

Using $A \rightarrow B \equiv \neg A \lor B$:

$\neg \mathrm{Vehicle}(x) \lor \neg \mathrm{EmergencyType}(x) \lor \mathrm{ClearPath}(x)$

Rule 2

Original:

$\mathrm{ClearPath}(x) \rightarrow \mathrm{GreenSignal}(x) \land \neg \mathrm{RedSignal}(x)$

This gives two CNF clauses:

$\neg \mathrm{ClearPath}(x) \lor \mathrm{GreenSignal}(x)$

and

$\neg \mathrm{ClearPath}(x) \lor \neg \mathrm{RedSignal}(x)$

Only the first one is needed for proving Proceed(CarA).

Rule 3

Original:

$Vehicle(x) \land Behind(x,y) \land GreenSignal(y) \rightarrow Proceed(x)$

CNF:

$\neg Vehicle(x) \lor \neg Behind(x,y) \lor \neg GreenSignal(y) \lor Proceed(x)$

Facts in CNF

$Vehicle(Ambulance)$ $EmergencyType(Ambulance)$ $Vehicle(CarA)$ $Behind(CarA,Ambulance)$

And negated goal:

$\neg Proceed(CarA)$

Step 2: Derive ClearPath(Ambulance)

Take Rule 1:

$\neg Vehicle(x) \lor \neg EmergencyType(x) \lor ClearPath(x)$

Substitute:

$\theta = \{x/Ambulance\}$

We obtain:

$\neg Vehicle(Ambulance) \lor \neg EmergencyType(Ambulance) \lor ClearPath(Ambulance)$

Resolve with:

$Vehicle(Ambulance)$

to obtain:

$\neg EmergencyType(Ambulance) \lor ClearPath(Ambulance)$

Resolve with:

$EmergencyType(Ambulance)$

Therefore:

$ClearPath(Ambulance)$

Step 3: Derive GreenSignal(Ambulance)

Rule 2:

$\neg ClearPath(x) \lor GreenSignal(x)$

Substitute x/Ambulance:

$\neg ClearPath(Ambulance) \lor GreenSignal(Ambulance)$

Resolve with:

$ClearPath(Ambulance)$

Therefore:

$GreenSignal(Ambulance)$

Step 4: Use Rule 3

Rule 3:

$\neg Vehicle(x) \lor \neg Behind(x,y) \lor \neg GreenSignal(y) \lor Proceed(x)$

Use substitution:

$\theta = \{x/CarA, y/Ambulance\}$

Giving:

$\neg Vehicle(CarA) \lor \neg Behind(CarA,Ambulance) \lor \neg GreenSignal(Ambulance) \lor Proceed(CarA)$

Resolve with:

$Vehicle(CarA)$

then:

$\neg Behind(CarA,Ambulance) \lor \neg GreenSignal(Ambulance) \lor Proceed(CarA)$

Resolve with:

$\text{Behind}(\text{CarA}, \text{Ambulance})$

giving:

$\neg \text{GreenSignal}(\text{Ambulance}) \lor \text{Proceed}(\text{CarA})$

Resolve with:

$\text{GreenSignal}(\text{Ambulance})$

Therefore:

$\text{Proceed}(\text{CarA})$

Step 5: Resolve with the Negated Goal

We initially added:

$\neg \text{Proceed}(\text{CarA})$

Now resolve:

$\text{Proceed}(\text{CarA})$

with

$\neg \text{Proceed}(\text{CarA})$

This produces the **empty clause**:

$\square$

The empty clause represents a contradiction.

Therefore:

$KB \models \text{Proceed}(\text{CarA})$

Hence, **Proceed(CarA) is logically entailed by the knowledge base.**

Resolution Chain

```
Vehicle(Ambulance)
        +
EmergencyType(Ambulance)
        |
        | Rule 1
        ↓
ClearPath(Ambulance)
        |
        | Rule 2
        ↓
GreenSignal(Ambulance)
        |
        | Rule 3 + Vehicle(CarA)
        |       + Behind(CarA,Ambulance)
        ↓
Proceed(CarA)
        |
        | Resolve with ¬Proceed(CarA)
        ↓
       □
```

(c) Handling Two Simultaneous Emergency Vehicles

Suppose there are two emergency vehicles:

Ambulance1

and

Ambulance2

The existing rules are **universally quantified**, so they can technically apply to both vehicles.

For example, if the following facts are added:

Vehicle(Ambulance1) EmergencyType(Ambulance1) Vehicle(Ambulance2) EmergencyType(Ambulance2)

Rule 1 can be instantiated twice:

{x/Ambulance1}

giving:

ClearPath(Ambulance1)

and

{x/Ambulance2}

giving:

ClearPath(Ambulance2)

Rule 2 then gives:

GreenSignal(Ambulance1)

and

GreenSignal(Ambulance2)

Is the current KB sufficient?

Logically, it is sufficient to recognize and assign a clear path/green signal to multiple emergency vehicles, provided the appropriate facts for each vehicle are present.

However, it is **not sufficient for safe traffic management when the two emergency vehicles interact or compete for the same road resources**. The current KB has no rules concerning:

- whether two emergency vehicles are approaching the same intersection;
- priority between emergency vehicles;
- whether their paths conflict;
- whether both can receive green signals simultaneously;
- which vehicle should proceed first;
- preventing collisions or conflicting movements.

Additional facts needed

For example:

Vehicle(Ambulance1) EmergencyType(Ambulance1) Vehicle(Ambulance2) EmergencyType(Ambulance2)

and, if applicable:

Approaching(Ambulance1,Intersection1) Approaching(Ambulance2,Intersection1) Conflict(Ambulance1,Ambulance2)

Additional rules needed

A priority rule could be represented as:

$$Conflict(x,y) \land HigherPriority(x,y) \rightarrow Hold(y)$$

A rule preventing simultaneous conflicting movement could be:

$$Conflict(x,y) \land Proceed(x) \rightarrow \neg Proceed(y)$$

A more realistic priority rule might be:

$$Emergency(x) \land Emergency(y) \land Conflict(x,y) \land HigherPriority(x,y) \rightarrow GreenSignal(x) \land RedSignal(y)$$

These rules would allow the agent to reason about **conflicts and priority**, rather than simply giving every emergency vehicle a green signal.

Final conclusion :

Current KB: sufficient for independent emergency vehicles

but

Current KB: insufficient for conflicting simultaneous emergencies

because it lacks **conflict detection, priority management, and mutually exclusive signal rules**.

Case Study 3 — Agricultural Expert Reasoning System

An agricultural advisory logical agent operates with the following propositional knowledge base (KB):

1) P1: SoilDry → IrrigationNeeded
2) P2: IrrigationNeeded ∧ CropWheat → ApplyDripMethod
3) P3: ¬ApplyDripMethod ∧ CropWheat → CropAtRisk
4) Facts: SoilDry = True, CropWheat = True

(a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication → move ¬ inward → distribute).

(b) Using Resolution Refutation, prove the theorem ‘ApplyDripMethod’. Negate the goal, combine with the KB in CNF, and derive the empty clause □. Draw the resolution proof tree.

(c) If the fact ‘SoilDry’ is removed from the KB, analyze step-by-step how logical conclusions change. Does ‘CropAtRisk’ still follow? Apply appropriate reasoning patterns to justify.

(a) Convert all sentences to CNF

P1: $S \rightarrow I$

Step 1: Eliminate implication

Using:

$A \rightarrow B \equiv \neg A \vee B$

we get:

$S \rightarrow I \equiv \neg S \vee I$

Step 2: Move negation inward

Already in the required form:

$\neg S \vee I$

Step 3: Distribute

No distribution is required because it is already a single clause.

Therefore:

$P1=(\neg S \vee I)$

P2: $I \wedge W \rightarrow D$

Step 1: Eliminate implication

$(I \wedge W) \rightarrow D \equiv \neg(I \wedge W) \vee D$

Step 2: Move negation inward

By De Morgan's law:

$\neg(I \wedge W) \equiv \neg I \vee \neg W$

Therefore:

$(\neg I \vee \neg W) \vee D$

or:

$\neg I \vee \neg W \vee D$

Step 3: Distribute

No further distribution is necessary.

Thus:

$P2=(\neg I \vee \neg W \vee D)$

P3: $\neg D \wedge W \rightarrow R$

Step 1: Eliminate implication

$(\neg D \wedge W) \rightarrow R \equiv \neg(\neg D \wedge W) \vee R$

Step 2: Move negation inward

Using De Morgan's law:

$\neg(\neg D \wedge W) \equiv \neg\neg D \vee \neg W$

Since:

$\neg\neg D = D$

we get:

$D \lor \neg W \lor R$

Step 3: Distribute

No distribution is necessary.

Therefore:

$P3 = (D \lor \neg W \lor R)$

Facts

The facts are:

$S = True\ W = True$

A positive fact is already a CNF unit clause:

$S\ W$

Complete CNF

Therefore, the complete KB in CNF is:

$C1{:}C2{:}C3{:}C4{:}C5{:}\neg S \lor I \neg I \lor \neg W \lor D D \lor \neg W \lor R S W$

(b) Resolution Refutation of ApplyDripMethod

The theorem to prove is:

D

Resolution refutation works by assuming the negation of the goal:

$\neg D$

We add this to the CNF KB and attempt to derive the empty clause □.

Thus, our clauses are:

$C1{:}\neg S \lor I\ C2{:}\neg I \lor \neg W \lor D\ C3{:}D \lor \neg W \lor R\ C4{:}S\ C5{:}W\ C6{:}\neg D$

Resolution Step 1: Derive I

Resolve C1 and C4:

$(\neg S \vee I), S$

The complementary literals are S and ¬S.

Therefore:

I

Resolution Step 2: Derive ¬W∨D

Resolve C2 with the newly derived I:

$(\neg I \vee \neg W \vee D), I$

Resolving on I:

$\neg W \vee D$

Resolution Step 3: Derive D

Resolve:

$\neg W \vee D$

with the fact:

W

The complementary literals are W and ¬W.

Therefore:

D

So we have derived:

ApplyDripMethod

Resolution Step 4: Contradiction

We originally added the negated goal:

$\neg D$

Now resolve:

D

with:

¬D

This gives:

□

The empty clause means a contradiction has been obtained.

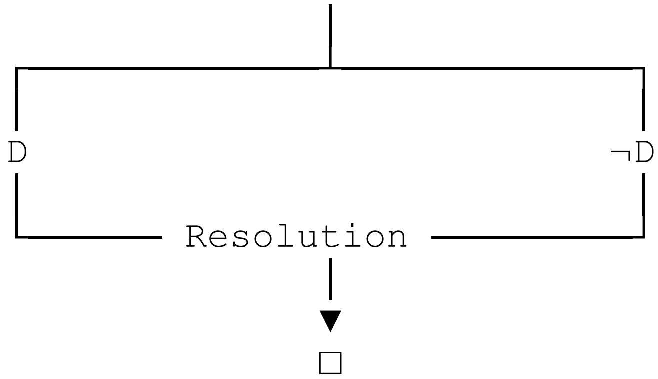
Therefore:

KB⊨ApplyDripMethod

So **ApplyDripMethod is logically entailed by the knowledge base.**

Resolution Proof Tree

```
                KB ∪ {¬D}
                    |
        ┌───────────┴───────────┐
        |                       |
    ¬S ∨ I                      S
        |                       |
        └───── Resolution ──────┘
                    |
                    ▼
                    I
                    |
        ┌───────────┴───────────┐
        |                       |
¬I ∨ ¬W ∨ D                     I
        |                       |
        └───── Resolution ──────┘
                    |
                    ▼
                 ¬W ∨ D
                    |
        ┌───────────┴───────────┐
        |                       |
                    W
        |                       |
        └───── Resolution ──────┘
                    |
                    ▼
                    D
```



Thus:

$\therefore$ApplyDripMethod

(c) Remove the Fact SoilDry

Now remove:

S

from the KB.

The remaining facts are:

W

The rules remain:

$\neg S \lor I \quad \neg I \lor \neg W \lor D \quad D \lor \neg W \lor R$

Step 1: Can we derive IrrigationNeeded?

P1 is:

$S \rightarrow I$

or in CNF:

$\neg S \lor I$

But S is no longer available as a fact.

Therefore, we cannot use P1 to derive I.

I cannot be derived

Notice that this does **not** mean:

$\neg I$

The absence of I is not the same as proving $\neg I$.

Step 2: Can we derive ApplyDripMethod?

P2 requires:

$I \wedge W$

We have:

W

but we do not have I.

Therefore:

D cannot be derived

Step 3: Can we derive CropAtRisk?

P3 is:

$\neg D \wedge W \rightarrow R$

To derive R, both conditions must be known:

$\neg D$

and

W

We know:

W

but there is **no fact or rule that derives ¬D**.

Therefore:

R cannot be derived

Why?

It is incorrect to reason:

"ApplyDripMethod cannot be proved, therefore ApplyDripMethod is false."

That would be the fallacy of **negation by failure**.

In classical propositional logic:

Unable to prove D⍰$\Rightarrow \neg D$

Consequently:

CropAtRisk does not follow

Final comparison

Proposition	**Original KB**	**After removing SoilDry**
SoilDry	True	Not known
IrrigationNeeded	**Derived**	Not derivable
CropWheat	True	True
ApplyDripMethod	**Derived**	Not derivable
ApplyDripMethod	Not derived	Not derived
CropAtRisk	Not derived	**Not derivable**

Final conclusion

With SoilDry present:

$S \Rightarrow I, I \wedge W \Rightarrow D$

so:

ApplyDripMethod

is proved.

After removing SoilDry, the chain is broken at IrrigationNeeded. However, this does **not** establish ¬ApplyDripMethod, so P3 cannot fire.

Therefore, CropAtRisk does not follow from the modified KB.

Case Study 4 — Wumpus World Knowledge Agent

An AI agent navigating the Wumpus World perceives: Stench at [1,2], Breeze at [1,1], Glitter at [2,2]. The agent uses a propositional KB with these rules:

1) Stench [1,2] → Wumpus is adjacent to cell [1,2]
2) Breeze [1,1] → Pit is adjacent to cell [1,1]
3) Glitter [x,y] → Gold is at [x,y]
4) ¬Wumpus [x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.

(b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Wumpus and Pit. Show each reasoning step precisely.

(c) Evaluate whether the path [1,1] → [1,2] → [2,2] is safe for the agent to collect the gold. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

a) Belief State and Modus Ponens

Step 1: Initial percepts

The agent initially knows:

$S_{1,2}$ $B_{1,1}$ $G_{2,2}$

Thus the initial belief state is:

$KB_0=\{S_{1,2},B_{1,1},G_{2,2}\}$

Step 2: Apply Rule 1 — Stench

Rule 1:

$S_{1,2}\rightarrow WumpusAdjacent(1,2)$

Since $S_{1,2}$ is true, by **Modus Ponens**:

$WumpusAdjacent(1,2)$

The cells adjacent to [1,2] are:

$[1,1], [1,3], [2,2]$

Therefore, the Wumpus must be in one of these cells:

$W1,1 \vee W1,3 \vee W2,2$

This is a **set of possibilities**, not a conclusion that the Wumpus is in all three cells.

Step 3: Apply Rule 2 — Breeze

Rule 2:

$B1,1 \rightarrow PitAdjacent(1,1)$

Since B1,1 is true:

$PitAdjacent(1,1)$

The cells adjacent to [1,1] are:

$[1,2], [2,1]$

Therefore:

$Pit1,2 \vee Pit2,1$

So the pit is possibly at either [1,2] or [2,1].

Step 4: Apply Rule 3 — Glitter

Rule 3:

$Glitter(x,y) \rightarrow Gold(x,y)$

For x=2,y=2:

$Glitter(2,2) \rightarrow Gold(2,2)$

Since glitter is perceived:

$Gold(2,2)$

Step 5: Apply Rule 4 — Safety

Rule 4:

$\neg Wumpus(x,y) \wedge \neg Pit(x,y) \rightarrow Safe(x,y)$

To apply this rule, the agent must know both:

$\neg Wumpus(x,y)$

and

$\neg Pit(x,y)$

But the given KB does not provide explicit negative facts about any particular cell.

Therefore, **Rule 4 cannot currently be applied**.

We cannot infer:

$Safe(1,1)$, $Safe(1,2)$, or $Safe(2,2)$

merely because safety has not been disproved.

Derived Belief State

The agent therefore knows:

$Stench(1,2)\ Breeze(1,1)\ Glitter(2,2)\ WumpusAdjacent(1,2)\ PitAdjacent(1,1)\ Gold(2,2)$

and the corresponding possible locations:

$Wumpus \in \{[1,1],[1,3],[2,2]\}\ Pit \in \{[1,2],[2,1]\}$

(b) Forward Chaining for Wumpus and Pit Locations

Forward chaining starts with the percepts and applies rules whose antecedents are satisfied.

Wumpus
Initial fact
$S_{1,2}$
Apply Rule 1
$S_{1,2} \rightarrow WumpusAdjacent(1,2)$

Therefore:

WumpusAdjacent(1,2)

The neighbors of [1,2] are:

[1,1],[1,3],[2,2]

Hence:

$W_{1,1} \lor W_{1,3} \lor W_{2,2}$

So the **possible Wumpus locations** are:

[1,1],[1,3],[2,2]

Pit

Initial fact

$B_{1,1}$

Apply Rule 2

$B_{1,1} \rightarrow PitAdjacent(1,1)$

Therefore:

PitAdjacent(1,1)

The neighbors of [1,1] are:

[1,2],[2,1]

Hence:

$Pit_{1,2} \lor Pit_{2,1}$

So the **possible pit locations** are:

[1,2],[2,1]

Gold

The glitter percept gives:

Glitter(2,2)

Applying Rule 3:

$Glitter(2,2) \rightarrow Gold(2,2)$

Therefore:

Gold(2,2)

Forward-Chaining Summary

```
Stench[1,2]
     │
     │ Rule 1
     ▼
Wumpus adjacent to [1,2]
     │
     ├── Wumpus may be at [1,1]
     ├── Wumpus may be at [1,3]
     └── Wumpus may be at [2,2]


Breeze[1,1]
     │
     │ Rule 2
     ▼
Pit adjacent to [1,1]
     │
     ├── Pit may be at [1,2]
     └── Pit may be at [2,1]


Glitter[2,2]
     │
     │ Rule 3
     ▼
Gold[2,2]
```

(c) Is the Path [1,1]→[1,2]→[2,2] Safe?

The proposed path is:

[1,1]→[1,2]→[2,2]

The agent wants to reach:

[2,2]

where it knows:

Gold(2,2)

However, knowing that gold is present does **not** establish that the cell is safe.

Check [1,2]

From the Breeze at [1,1]:

$B_{1,1} \rightarrow Pit_{1,2} \vee Pit_{2,1}$

Therefore:

$Pit_{1,2}$ is possible

Thus the cell [1,2] **cannot be proven safe**.

Check [2,2]

From the Stench at [1,2]:

$S_{1,2} \rightarrow W_{1,1} \vee W_{1,3} \vee W_{2,2}$

Therefore:

$W_{2,2}$ is possible

So the destination [2,2] could contain the Wumpus.

Although:

$Gold(2,2)$

is known, there is no rule saying that a cell containing gold is safe.

Can Rule 4 establish safety?

Rule 4 requires:

$\neg Wumpus(x,y) \wedge \neg Pit(x,y) \rightarrow Safe(x,y)$

But we have no explicit facts such as:

$\neg W_{1,2}$ $\neg Pit_{1,2}$

or:

$\neg W_{2,2}$ $\neg Pit_{2,2}$

Therefore, we cannot derive:

Safe(1,2)

or:

Safe(2,2)

Final Conclusion

The path:

[1,1]→[1,2]→[2,2]

is **not logically proven safe**.

Specifically:

- [1,2] may contain a **Pit.**
- [2,2] may contain the **Wumpus**.
- [2,2] does contain the **Gold**, but gold does not imply safety.
- Rule 4 cannot establish safety because explicit negative information about Wumpus and Pit locations is unavailable.

Therefore:

The agent should not conclude that this path is safe.

More precisely, the correct logical conclusion is **"safety is unknown/not guaranteed," rather than "the path is definitely dangerous."**

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand